

Backend Developer.

From zero to a production-shaped Python backend, in five days.

"Six agents. Five days. Zero hallucination."

A complete, paste-ready Antigravity build guide for the Backend Agents track of the Bulao project — AI Seekho 2026.

Every prompt is self-contained. Every file path, dependency, schema, and convention is spelled out. Antigravity will not need to guess.

How to use this document:

1. Read 'Your role' and 'System context' first (sections 01-02).
2. On each build day, paste the day's prompt into Antigravity.
3. Verify with the day's checklist before moving on.
4. Reference the appendices when Antigravity asks for clarification.

01 · YOUR ROLE

You own backend agents.

You are the **Backend Developer** on the Bulao build team. There are four teammates total — Product & Urdu, Mobile, Backend, Infrastructure & Trace. This document is yours alone. Each teammate has their own pack. Do not try to do the others' work; coordinate handoffs through WhatsApp + a shared Drive.

What you own

- The entire backend codebase skeleton (backend/ directory)
- Intent Agent — confidence + complexity classification
- Discovery Agent — filtering, scheduling, alternate slots
- Ranking Agent — six-factor scoring with Urdu reasoning
- Pricing Agent — transparent line-item quote
- The 100-row mock provider dataset (providers.json)
- Pydantic schemas for Intent, ProviderCandidate, PriceQuote
- The Intent Agent eval harness

What you do *not* own

- Booking, Follow-up & Dispute agents (Backend Workflows owns these)
- FastAPI orchestrator endpoints (Backend Workflows owns)
- Flutter / mobile code
- Cloud Run console operations (Infra owns)

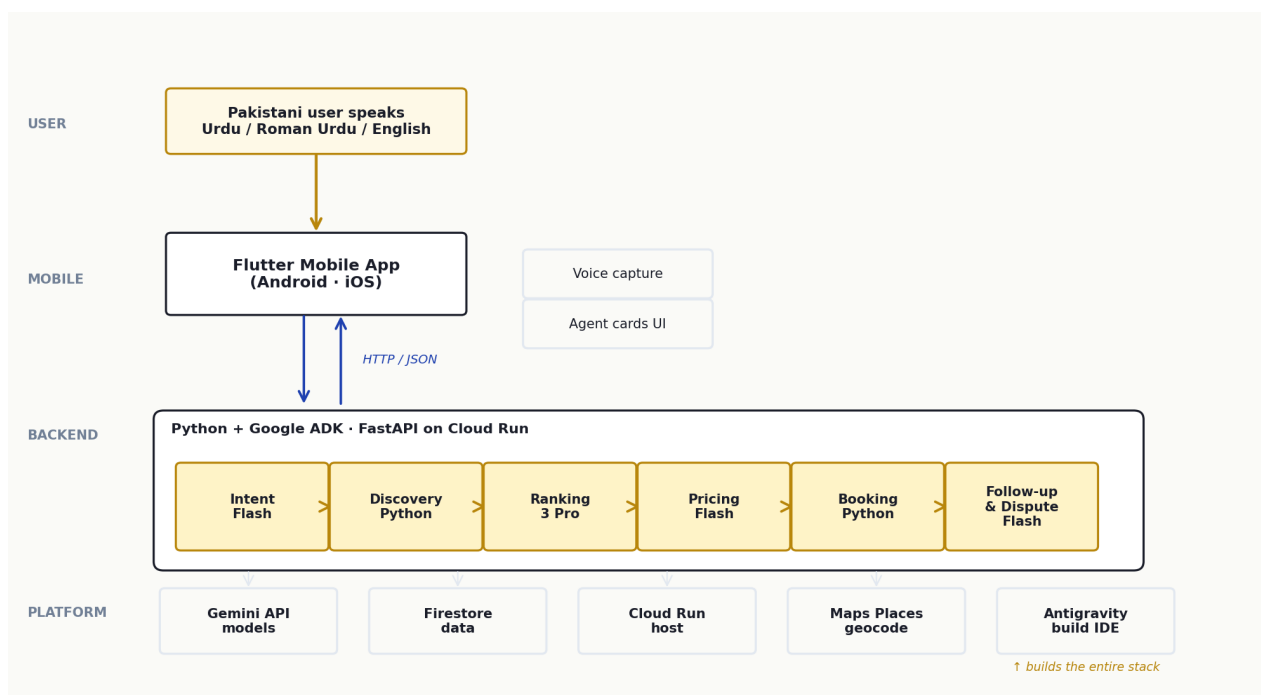
If you find yourself doing work outside the 'what you own' list, stop and ask in the team WhatsApp. Five days is short. Stay in your lane and move fast.

02 · SYSTEM CONTEXT

What Bulao is, end to end.

Bulao is a voice-first, Urdu-native AI service orchestrator for Pakistan's informal economy — plumbers, electricians, AC technicians, beauticians, tutors. A user presses a big button, speaks in Urdu / Roman Urdu / English, and within ninety seconds watches **six AI agents** extract their intent, find providers, rank them, produce a transparent price quote, book a slot, and follow up through completion — including fair dispute resolution when something goes wrong.

We are building this for the AI Seekho 2026 Antigravity Hackathon (Challenge 2). Five days, four teammates, one demo on May 20. Regional finals May 25-26, finals June 7 in Islamabad. Prize pool PKR 2.5M.



Three layers: a Flutter mobile app captures voice and renders six AgentCards; a Python backend on Cloud Run runs the six-agent pipeline using Google ADK and Gemini; Antigravity is the IDE that builds the stack — not part of the runtime, but the source of the 25%-weighted Agent Trace deliverable.

The six agents

Every teammate must know what each agent does, what it takes in, and what it produces. This is the shared mental model.



Your relationship to the six agents

You **build the intelligence agents**. Four of them: Intent, Discovery, Ranking, and Pricing. Each is a Python module that loads its system prompt from disk, calls Gemini through Google ADK (or runs as pure Python), parses the JSON response into a Pydantic model, and returns it. You also build the 100-row mock provider dataset, the Pydantic schemas that the whole backend shares, and the eval harness for the Intent Agent.

Key handoffs you must respect

FROM	Day	TO	Deliverable
You	1 evening	Workflows	Scaffold at backend/ — poetry run uvicorn working
You	1 evening	Mobile+Infra	Stub /orchestrate returning all 6 outputs as mock JSON
Product	2 morning	You	First 25 of intent_examples.jsonl
You	2 evening	Workflows	Real intent_agent.py + discovery_agent.py importable
You	3 noon	Workflows	Real ranking_agent.py + pricing_agent.py importable
You	5 morning	All	All 4 agents pass their tests; eval_report.md final

03 · DAY 1 · BEFORE MAY 15

Day 0 (May 14) · setup

Install Python 3.11, Poetry, Docker, gcloud CLI, and Antigravity. Complete every item before May 15 morning — environment failures on Day 1 cost the team hours it doesn't have.

Setup · install everything you need

This is not an Antigravity prompt — do these by hand before May 15. Antigravity cannot install software on your laptop for you.

PROMPT · LOCAL ENVIRONMENT CHECKLIST (RUN YOURSELF)

Complete these on your laptop by end of Day 0 (May 14):

1. Install Python 3.11 (NOT 3.12; google-adk's most stable line is on 3.11).
 - macOS: brew install python@3.11
 - Linux: sudo apt install python3.11 python3.11-venv python3.11-dev
 - Windows: download from python.org, check "Add to PATH"
2. Install Poetry (Python dependency manager):
 - curl -sSL https://install.python-poetry.org | python3 -
 - poetry --version # should show >= 1.7
3. Install Docker Desktop and verify it runs:
 - docker --version
 - docker run --rm hello-world
4. Install gcloud CLI:
 - https://cloud.google.com/sdk/install
 - gcloud auth login (use the same email as the team's GCP account)
 - gcloud config set project bulao-hackathon
 - gcloud auth application-default login
5. Install Google Antigravity (<https://antigravity.google/download>).
 - Launch, sign in with the same Gmail.
 - Settings -> Agent panel:
 - Mode = Agent-assisted development
 - Artifact Review Policy = Asks for Review
 - Terminal Command Auto Execution = Request Review
 - Enable Terminal Sandbox = ON
 - Default model = Gemini 3 Pro
6. Get your Gemini API key from <https://aistudio.google.com> -> Get API key.
Create a key in a project named bulao-hackathon. Copy it; you will paste it into Secret Manager (Infra teammate will do this) and into your local .env file.
7. Clone the team repo (Infra will share the URL).
 - git clone
 - cd bulao
 - ls # you should see mobile/ backend/ docs/ at minimum

If anything fails, post in WhatsApp BEFORE Day 1 starts. Don't burn build hours on environment setup.

End-of-day verification

- `python3.11 --version` shows 3.11.x.
- `poetry --version` shows ≥ 1.7 .
- `docker run --rm hello-world` works.
- `gcloud projects list` shows bulao-hackathon.
- Antigravity launches; you can spawn a new Manager View workspace.
- You have your Gemini API key saved in your password manager.
- The team repo is cloned and `backend/` directory exists.

04 · DAY 2 · MAY 15

Day 1 · scaffold the backend skeleton

Today you create the entire backend skeleton: six placeholder agents, the ADK SequentialAgent orchestrator, a working /orchestrate endpoint returning stub JSON, the 100-row mock provider dataset, and a passing test suite. By 6pm, the Backend Workflows teammate can start wiring their endpoints on top of your scaffold.

The scaffolding prompt (paste in Antigravity Manager View)

This single prompt builds the entire backend skeleton. Antigravity will produce a Plan Artifact first — review every file before approving. The prompt is self-contained: file paths, dependencies, schemas, and verification commands are all spelled out.

PROMPT · DAY 1 — SCAFFOLD THE BACKEND

ROLE: You are setting up the backend repository for "Bulao" - a six-agent voice-first Urdu service-orchestrator for Pakistan, built for AI Seekho 2026 Antigravity Hackathon.

WHAT TO BUILD TODAY: A scaffolded FastAPI backend with 6 placeholder agents, a working /orchestrate endpoint returning hardcoded mock JSON, a /health endpoint, a Dockerfile, and a passing test suite. Deployable to Cloud Run by end of day.

EXACT DIRECTORY STRUCTURE (create every file):


```
backend/  
README.md  
pyproject.toml  
Dockerfile  
.env.example  
.gitignore  
app/  
__init__.py  
main.py  
config.py  
orchestrator.py  
models.py  
agents/  
__init__.py  
intent_agent.py  
discovery_agent.py  
ranking_agent.py  
pricing_agent.py  
booking_agent.py  
followup_agent.py  
prompts/  
intent.md (empty placeholder; Product owns this)  
ranking.md  
pricing.md  
booking.md  
followup.md  
data/  
providers.json (empty array [] for now)  
intent_examples.jsonl (empty file)  
tools/  
__init__.py  
tests/  
__init__.py  
test_health.py  
test_orchestrate.py  
scripts/  
eval_intent.py (stub for now)  
  
PYTHON: 3.11
```

DEPENDENCIES (in pyproject.toml):

- python = "^3.11"
- fastapi = "^0.110"
- "uvicorn[standard]" = "^0.27"
- google-adk = "^0.5"
- google-genai = "^0.3"
- pydantic = "^2.6"
- pydantic-settings = "^2.2"
- google-cloud-firestore = "^2.15"
- google-cloud-storage = "^2.14"
- reportlab = "^4.0"
- httpx = "^0.27"
- structlog = "^24.1"
- (dev) pytest = "^8.0"
- (dev) pytest-asyncio = "^0.23"
- (dev) ruff = "^0.4"

PYDANTIC MODELS (in app/models.py) — define ALL of these exactly:

```
class Intent(BaseModel):
    service_type: Literal["plumber", "electrician", "ac_technician", "geyser_technician", "carpenter", "painter", "beautician", "tutor", "appliance_repair", "gas_leak_specialist"]
    location: Optional[str] = None
    city: str = "Islamabad"
    time_window: str
    urgency: Literal["emergency", "high", "normal", "flexible"]
    job_complexity: Literal["basic", "intermediate", "complex"]
    specialization_hint: Optional[str] = None
    gender_preference: Literal["male", "female", "any"] = "any"
    budget_range: Optional[Tuple[int, int]] = None
    raw_notes: Optional[str] = None
    confidence: float # 0.0 to 1.0
    clarification_question: Optional[str] = None
```

```

class ProviderCandidate(BaseModel):
    id: str
    name: str
    service_categories: List[str]
    specializations: List[str] = []
    distance_km: float
    neighborhood: str
    rating: float
    completed_jobs_in_area: int
    on_time_score: float
    cancellation_rate: float
    review_recency_days: int
    risk_score: float
    current_workload: float
    availability_status: Literal["available_now", "next_slot_within_2h", "next_slot_today", "tomorrow_or_later"]
    next_slot: str # ISO datetime
    base_visit_fee_pkr: int
    rate_per_hour_pkr: int
    gender: Literal["male", "female"]
    years_experience: int
    languages: List[str] = []
    verified: bool = True
    phone_masked: str
    alternate_slot_reason: Optional[str] = None # only on alternates

class DiscoveryResult(BaseModel):
    candidates: List[ProviderCandidate]
    alternates: List[ProviderCandidate] = []
    no_match_reason: Optional[str] = None

class FactorScores(BaseModel):
    distance: float
    availability: float
    rating: float
    on_time: float
    specialization: float
    risk: float
    total: float

class RankingResult(BaseModel):
    recommended_id: Optional[str]
    top_three_ids: List[str] = []
    factor_scores: Dict[str, FactorScores] = {}
    reasoning_english: str = ""
    reasoning_urdu: str = ""
    tradeoffs: str = ""
    confidence: Literal["high", "medium", "low"] = "medium"
    error: Optional[str] = None

```

```

class PriceLineItem(BaseModel):
    label_english: str
    label_urdu: str
    amount_pkr: int # negative for discounts
    kind: Literal["base","addon","multiplier","discount","surge"]

class PriceQuote(BaseModel):
    quote_id: str
    line_items: List[PriceLineItem]
    subtotal_pkr: int
    estimated_total_pkr: int
    estimated_range_pkr: Tuple[int,int]
    explanation_english: str
    explanation_urdu: str
    fairness_note: str
    expires_at: str # ISO

class BookingLifecycle(BaseModel):
    confirmed_at: Optional[str] = None
    en_route_at: Optional[str] = None
    arrived_at: Optional[str] = None
    in_progress_at: Optional[str] = None
    completed_at: Optional[str] = None
    cancelled_at: Optional[str] = None

class Booking(BaseModel):
    booking_id: str
    user_id: str
    provider_id: str
    service_type: str
    location: str
    city: str
    scheduled_time: str
    status: Literal["confirmed","en_route","arrived","in_progress","completed","cancelled","disputed"] =
    "confirmed"
    lifecycle: BookingLifecycle
    accepted_quote: PriceQuote
    intent_snapshot: Intent
    ranking_snapshot: Optional[RankingResult] = None
    receipt_url: Optional[str] = None
    confirmation_message_english: str = ""
    confirmation_message_urdu: str = ""

class FollowupResult(BaseModel):
    mode: Literal["reminder","checkin","dispute"]
    english: str = ""
    urdu: str = ""
    cta: Literal["rate","contact_provider","none"] = "none"
    classification: Optional[str] = None
    resolution: Optional[str] = None
    refund_pkr: int = 0
    escalate_to_human: bool = False

```

```

class OrchestrateRequest(BaseModel):
    text: str
    user_id: str
    user_location: Optional[Tuple[float,float]] = None # (lat, lng)

class OrchestrateResponse(BaseModel):
    intent: Intent
    discovery: DiscoveryResult
    ranking: RankingResult
    pricing: PriceQuote
    booking_preview: Dict # booking-shaped; not committed until /book
    followup_planned: Dict # reminder timing only

CONFIG (app/config.py) using pydantic-settings:
class Settings(BaseSettings):
    GEMINI_API_KEY: str
    GOOGLE_CLOUD_PROJECT: str = "bulao-hackathon"
    DEMO_MODE: bool = False
    FIRESTORE_EMULATOR_HOST: Optional[str] = None
    LOG_LEVEL: str = "INFO"

class Config:
    env_file = ".env"
    env_file_encoding = "utf-8"

settings = Settings()

FASTAPI APP (app/main.py):
- Use lifespan context to init logger + ADK client at startup
- CORS middleware allowing all origins (hackathon scope)
- Global exception handler returning JSON {error: str, code: int}
- Three endpoints:
GET /health -> {status: "ok", version: "0.1.0", commit: }
POST /orchestrate -> calls orchestrator.run_pipeline(req); returns OrchestrateResponse
POST /book -> stub for Day 3; returns 501 Not Implemented for now

ORCHESTRATOR (app/orchestrator.py):
- async def run_pipeline(req: OrchestrateRequest) -> OrchestrateResponse
- For Day 1, returns hardcoded mock JSON matching the schemas (use realistic Pakistani names).
- Specifically: intent for "plumber chahiye" -> service_type=plumber, location=null, urgency=high,
confidence=0.78
- Use one consistent mock provider "Ali Plumbing Works" in G-13

AGENT PLACEHOLDERS (app/agents/*.py):
- Each file exports: async def run(input_data: ) ->
- Day 1 implementation: just return hardcoded mock data matching the schema
- Add a TODO comment: "# Day N: Replace with real Gemini call. See app/prompts/.md."

LOGGING (set up in app/main.py lifespan):
- structlog with JSON renderer to stdout
- Bind {service: "bulao-backend", env: }
- Each agent's run() logs: {"event": "agent_start|"agent_end", "agent": , "duration_ms": N}

```

DOCKERFILE (multi-stage):

```
# Stage 1: build
FROM python:3.11-slim AS builder
RUN pip install poetry==1.7.1
COPY pyproject.toml poetry.lock* ./
RUN poetry config virtualenvs.in-project true && poetry install --no-root --without dev
```

```
# Stage 2: runtime
FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /.venv /.venv
COPY app ./app
ENV PATH="/app/.venv/bin:$PATH" PORT=8080
EXPOSE 8080
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

.env.example:

```
GEMINI_API_KEY=paste-your-key-here
GOOGLE_CLOUD_PROJECT=bulao-hackathon
DEMO_MODE=false
LOG_LEVEL=INFO
```

.gitignore: standard Python (.venv, __pycache__, *.pyc) + .env + dist/ + .pytest_cache + .ruff_cache

README.md: 4-section skeleton — Overview, Run locally, Deploy, Architecture (one paragraph each)

TESTS (app/tests/):

- test_health.py: assert GET /health returns 200 with {"status":"ok"}
- test_orchestrate.py: POST /orchestrate with {text:"plumber chahiye", user_id:"u_test"} returns 200 with all 6 keys present, intent.service_type == "plumber", confidence between 0 and 1

VERIFY (run these and confirm they all pass):

1. cd backend && poetry install
2. poetry run uvicorn app.main:app --reload --port 8080 &
3. curl http://localhost:8080/health -> {"status":"ok","version":"0.1.0"}
4. curl -X POST http://localhost:8080/orchestrate -H 'Content-Type: application/json' -d '{"text":"plumber chahiye","user_id":"u_test"}' -> JSON with intent, discovery, ranking, pricing, booking_preview, followup_planned
5. poetry run pytest -> 2 passed
6. docker build -t bulao-backend . -> succeeds
7. docker run --rm -p 8080:8080 -e GEMINI_API_KEY=fake bulao-backend & sleep 5 && curl http://localhost:8080/health -> {"status":"ok"}

OUTPUT WHEN DONE:

- All files created exactly as specified.
- Plan Artifact shown to the developer for review BEFORE any code is written.
- Final tool call summary lists every file created with its byte count.

If you need to deviate from this spec for any reason, STOP and ask the developer in the Plan Artifact. Do not assume.

End-of-day verification

- poetry run pytest shows 2 passed.
 - curl localhost:8080/health returns {"status":"ok","version":"0.1.0"}.
 - curl -X POST localhost:8080/orchestrate with a sample body returns all 6 agent keys.
 - docker build . succeeds.
 - Backend repo committed with the full file tree; Mobile teammate can git pull.
 - Infra teammate has the URL pattern needed to deploy this image.
 - Screenshot of Antigravity's Plan Artifact saved to docs/antigravity-trace/day1/.
- *Once the scaffold is live, ping Backend Workflows in WhatsApp with the exact directory structure. Any file you add after tonight that they import from costs them time.*

05 · DAY 3 · MAY 16

Day 2 · Intent + Discovery go live

Intent Agent learns Urdu; Discovery Agent learns to find providers. By 8pm, a typed Pakistani sentence produces a real provider list from the mock database.

Morning · build the real Intent Agent

Replace the Day 1 stub with a Gemini-powered Intent Agent. The Product teammate delivers the system prompt and the 50-example dataset; you wire them in, add the eval harness, and write tests.

PROMPT · DAY 2 — INTENT AGENT (WITH CONFIDENCE + COMPLEXITY)

ROLE: You are upgrading the Intent Agent from Day 1's hardcoded stub to a real Gemini-powered agent that parses Urdu/Roman Urdu/English service requests.

FILES TO EDIT:

- app/agents/intent_agent.py (replace stub with real implementation)
- app/prompts/intent.md (paste in the system prompt — Product team will provide)
- app/tests/test_intent_agent.py (NEW — eval against intent_examples.jsonl)
- scripts/eval_intent.py (NEW — CLI for the Product teammate to iterate)

CONTEXT THE PRODUCT TEAMMATE HAS ALREADY DELIVERED:

- app/data/intent_examples.jsonl — 25-50 examples, each line: {"input": str, "expected": {}}
- app/prompts/intent.md — full system prompt (paste it into the file as-is; do not edit)

WHAT TO IMPLEMENT IN intent_agent.py:

```
import json
from pathlib import Path
from google.adk.agents import LlmAgent
from google.adk.models import Gemini
from app.config import settings
from app.models import Intent

_PROMPT_PATH = Path(__file__).parent.parent / "prompts" / "intent.md"
_SYSTEM_PROMPT = _PROMPT_PATH.read_text(encoding="utf-8")

_agent = LlmAgent(
    name="intent",
    model=Gemini(model_name="gemini-flash-latest", temperature=0.1, api_key=settings.GEMINI_API_KEY),
    system_instruction=_SYSTEM_PROMPT,
    response_mime_type="application/json",
)

async def run(user_text: str) -> Intent:
    """Extract structured Intent from user text. Falls back gracefully on parse failure."""
    # Log start
    log.info("agent_start", agent="intent", input_len=len(user_text))
    t0 = time.monotonic()
```



```

try:
    raw = await _agent.invoke(user_text)
except Exception as e:
    log.error("agent_llm_failure", agent="intent", error=str(e))
    return _fallback_intent(user_text, reason="llm_failure")

# Parse JSON
try:
    data = json.loads(raw)
except json.JSONDecodeError:
    # Retry once with explicit JSON-only reminder
    try:
        raw2 = await _agent.invoke(user_text + "\n\nReturn ONLY valid JSON. No markdown.")
        data = json.loads(raw2)
    except Exception:
        log.error("agent_json_parse_failure", agent="intent", raw_preview=raw[:200])
        return _fallback_intent(user_text, reason="json_parse")

# Validate against schema
try:
    intent = Intent.model_validate(data)
except ValidationError as e:
    log.error("agent_schema_violation", agent="intent", errors=e.errors()[:3])
    return _fallback_intent(user_text, reason="schema_violation")

log.info("agent_end", agent="intent", duration_ms=int((time.monotonic()-t0)*1000),
        service_type=intent.service_type, confidence=intent.confidence)
return intent

def _fallback_intent(user_text: str, reason: str) -> Intent:
    """Best-effort keyword-based fallback when the LLM fails."""
    keywords = {
        "plumber": ["plumber", "pani", "leak", "nal", "pipe", "ghouse"],
        "electrician": ["electrician", "bijli", "light", "wiring", "switch"],
        "ac_technician": ["ac", "air condition", "inverter", "split", "window"],
        "geyser_technician": ["geyser", "heater", "garam pani"],
        "carpenter": ["carpenter", "lakdi", "wood", "furniture", "cupboard"],
        "painter": ["paint", "painter", "rang", "color"],
        "beautician": ["beautician", "makeup", "bridal", "facial", "hair"],
        "tutor": ["tutor", "teacher", "ustaad", "math", "english"],
        "appliance_repair": ["fridge", "washing machine", "oven", "appliance"],
        "gas_leak_specialist": ["gas leak", "gas smell"],
    }
    text_lower = user_text.lower()
    service = "plumber" # safe default
    for svc, kws in keywords.items():
        if any(kw in text_lower for kw in kws):
            service = svc
    break

```

```

return Intent(
    service_type=service,
    location=None,
    city="Islamabad",
    time_window="flexible",
    urgency="normal",
    job_complexity="basic",
    gender_preference="any",
    confidence=0.3,
    clarification_question="Aap ne kya kaam karwana hai? Sector ya area ka naam bhi batayein.",
    raw_notes=f"fallback:{reason}",
)

```

WIRE INTO ORCHESTRATOR (app/orchestrator.py):

- Replace the hardcoded intent block with: `intent = await intent_agent.run(req.text)`
- If `intent.confidence < 0.7`, set a flag "needs_clarification" in the response and SKIP discovery (return early with just the intent + the clarification_question).
- Otherwise proceed to discovery (still stub for now; Discovery is the next prompt).

EVAL HARNESS (scripts/eval_intent.py):

- CLI: `poetry run python scripts/eval_intent.py [--prompt-file PATH]`
- Loads `app/data/intent_examples.jsonl`
- For each example: runs `intent_agent.run(example["input"])`, compares to `example["expected"]`
- Reports: service_type accuracy, job_complexity accuracy, confidence-vs-threshold accuracy
- Shows a confusion matrix for service_type
- Lists the 10 worst-failing examples with the expected vs actual
- Exit code 0 if service_type accuracy ≥ 0.90 AND job_complexity accuracy ≥ 0.85 ; else 1

TESTS (app/tests/test_intent_agent.py):

- pytest fixture loads `intent_examples.jsonl`
- Tests:
 - a) service_type accuracy $\geq 90\%$
 - b) job_complexity accuracy $\geq 85\%$
 - c) For examples with `confidence_min`: actual confidence $\geq$ confidence_min - 0.1
 - d) For examples with `needs_clarification=True`: actual confidence < 0.7 AND clarification_question is not None
- Mark with `@pytest.mark.asyncio`
- If `GEMINI_API_KEY` is not set, skip with a clear message

VERIFY:

1. `poetry run pytest app/tests/test_intent_agent.py` -> all pass (assumes Product has delivered the dataset + prompt)
2. `poetry run python scripts/eval_intent.py` -> prints accuracy + confusion matrix; exit code 0
3. `curl -X POST localhost:8080/orchestrate -d '{"text":"Mujhe G-13 mein kal subah AC theek karwana hai","user_id":"u_test"}'` returns `intent.service_type=="ac_technician", location=="G-13", time_window=="tomorrow_morning", confidence > 0.85`
4. `curl -X POST localhost:8080/orchestrate -d '{"text":"mere ghar pe plumber bhejo","user_id":"u_test"}'` returns confidence < 0.7 AND clarification_question populated (asking for location)

If accuracy is below target, stop and report which examples failed. Do NOT silently weaken the assertions.

Afternoon · build the Discovery Agent

Discovery is the only agent without an LLM — pure Python. It filters, applies the 30-minute travel buffer, and triggers the alternate-slot path when fewer than three candidates match.

PROMPT · DAY 2 — DISCOVERY AGENT (NO LLM)

ROLE: You are implementing the Discovery Agent — the only agent in the system that does NOT use an LLM. It is pure Python: a filter, scheduler, and pre-ranker over the mock provider database.

FILES TO EDIT/CREATE:

- app/agents/discovery_agent.py (replace stub)
- app/data/providers.json (generate 100 entries — see schema below)
- app/data/neighborhoods.json (NEW — 14 entries with lat/long centers)
- app/tests/test_discovery_agent.py (NEW)
- scripts/gen_providers.py (NEW — one-time script that generates providers.json)

GENERATE THE 100 MOCK PROVIDERS first (scripts/gen_providers.py):

```
import json, random
from pathlib import Path
from datetime import datetime, timedelta

random.seed(42) # reproducible

NEIGHBORHOODS = {
    "F-6": (33.7218, 73.0667), "F-7": (33.7176, 73.0531), "F-8": (33.7068, 73.0337),
    "F-10": (33.6905, 73.0205), "F-11": (33.6850, 73.0095), "G-7": (33.6979, 73.0683),
    "G-9": (33.6892, 73.0337), "G-10": (33.6750, 73.0250), "G-11": (33.6650, 73.0150),
    "G-13": (33.6584, 73.0479), "Bahria Town Ph 4": (33.5210, 73.0900),
    "DHA Phase 2": (33.5100, 73.1500), "Saddar Rawalpindi": (33.5950, 73.0480),
    "Pindi Cantt": (33.5810, 73.0490),
}

SERVICE_POOL = ["plumber", "electrician", "ac_technician", "geyser_technician",
    "carpenter", "painter", "beautician", "tutor", "appliance_repair", "gas_leak_specialist"]
SPECIALIZATIONS = {
    "ac_technician": ["inverter_ac", "split_ac", "window_ac", "central_ac"],
    "plumber": ["geyser_leak", "tap_replacement", "pipe_burst", "drain_cleaning"],
    "electrician": ["house_wiring", "switchboard", "ups_install", "fan_install"],
    "beautician": ["bridal", "engagement", "party_makeup", "hair_styling", "facial"],
    "tutor": ["o_level_math", "a_level_business", "quran", "english"],
}

FIRST_NAMES_M = ["Ali", "Ahmed", "Hassan", "Bilal", "Usman", "Fahad", "Tariq", "Imran", "Saad", "Zain"]
FIRST_NAMES_F = ["Ayesha", "Sana", "Fatima", "Hina", "Maria", "Nida", "Sara", "Zoya", "Iqra", "Mahnoor"]
LAST_NAMES = ["Khan", "Ali", "Hussain", "Ahmed", "Malik", "Sheikh", "Qureshi", "Butt", "Mughal", "Shah"]
```

```

def gen_provider(i: int) -> dict:
    neighborhood = random.choice(list(NEIGHBORHOODS.keys()))
    lat, lng = NEIGHBORHOODS[neighborhood]
    lat += random.uniform(-0.01, 0.01); lng += random.uniform(-0.01, 0.01)
    gender = random.choice(["male", "female"])
    fname = random.choice(FIRST_NAMES_M if gender == "male" else FIRST_NAMES_F)
    lname = random.choice(LAST_NAMES)
    primary_service = random.choice(SERVICE_POOL)
    services = [primary_service]
    if random.random() < 0.3:
        services.append(random.choice([s for s in SERVICE_POOL if s != primary_service]))
    specs = SPECIALIZATIONS.get(primary_service, [])
    chosen_specs = random.sample(specs, k=min(2, len(specs))) if specs else []
    rating = round(random.gauss(4.3, 0.35), 1); rating = max(3.5, min(5.0, rating))
    # Available slots: 2-5 in the next 7 days
    slots = []
    now = datetime.now()
    for _ in range(random.randint(2,5)):
        day_offset = random.randint(0,6)
        hour = random.choice([9,10,11,14,15,16,17])
        slots.append((now + timedelta(days=day_offset)).replace(hour=hour, minute=0, second=0,
            microsecond=0).isoformat() + "+05:00")
    return {
        "id": f"prov_{i:03d}",
        "name": f"{fname} {lname} {primary_service.replace('_', ' ').title()}",
        "service_categories": services,
        "specializations": chosen_specs,
        "lat": round(lat,6), "lng": round(lng,6),
        "neighborhood": neighborhood, "city": "Islamabad" if "Rawalpindi" not in neighborhood and "Pindi" not in
        neighborhood else "Rawalpindi",
        "rating": rating,
        "completed_jobs": random.randint(20, 250),
        "completed_jobs_by_neighborhood": {neighborhood: random.randint(5,80)},
        "on_time_score": round(random.uniform(0.78, 0.99), 2),
        "cancellation_rate": round(random.uniform(0.01, 0.12), 2),
        "review_recency_days": random.randint(1, 60),
        "risk_score": round(random.uniform(0.03, 0.25), 2),
        "current_workload": round(random.uniform(0.1, 0.85), 2),
        "available_slots": sorted(slots),
        "phone": f"+9233{random.randint(10000000, 99999999)}",
        "gender": gender,
        "years_experience": random.randint(1, 18),
        "languages": ["urdu", "english"] if random.random() > 0.3 else ["urdu"],
        "base_visit_fee_pkr": random.choice([400, 500, 600, 750, 1000]),
        "rate_per_hour_pkr": random.choice([800, 1000, 1200, 1500, 2000]),
        "price_range_pkr": [random.randint(800, 1500), random.randint(3000, 8000)],
        "verified": random.random() > 0.1,
    }

```

```

if __name__ == "__main__":
    providers = [gen_provider(i+1) for i in range(100)]
    Path("app/data/providers.json").write_text(json.dumps(providers, indent=2, ensure_ascii=False))
    Path("app/data/neighborhoods.json").write_text(json.dumps({k:{"lat":v[0],"lng":v[1]} for k,v in
    NEIGHBORHOODS.items()}), indent=2))
    print(f"Generated {len(providers)} providers across {len(NEIGHBORHOODS)} neighborhoods.")

```

RUN IT ONCE: poetry run python scripts/gen_providers.py

NOW IMPLEMENT discovery_agent.py:

```

import json, math
from pathlib import Path
from typing import Optional, Tuple
from app.models import Intent, ProviderCandidate, DiscoveryResult

_DATA_PATH = Path(__file__).parent.parent / "data" / "providers.json"
_NEIGH_PATH = Path(__file__).parent.parent / "data" / "neighborhoods.json"
_PROVIDERS = json.loads(_DATA_PATH.read_text())
_NEIGHBORHOODS = json.loads(_NEIGH_PATH.read_text())

def haversine_km(lat1, lng1, lat2, lng2):
    R = 6371
    p1, p2 = math.radians(lat1), math.radians(lat2)
    dp = math.radians(lat2-lat1); dl = math.radians(lng2-lng1)
    a = math.sin(dp/2)**2 + math.cos(p1)*math.cos(p2)*math.sin(dl/2)**2
    return 2*R*math.asin(math.sqrt(a))

```

ALGORITHM (in order):

1. SERVICE FILTER: keep providers where intent.service_type is in service_categories.
2. GENDER FILTER: if intent.gender_preference != "any", keep matching gender.
3. COMPLEXITY FILTER:
 - "complex" -> require years_experience >= 5 AND at least one specialization matching intent.specialization_hint (if set)
 - "intermediate" -> require years_experience >= 2
 - "basic" -> no filter
4. LOCATION DISTANCE: compute distance_km using user_location if provided, else neighborhood center of intent.location. If intent.location is None, do not filter by distance.
5. TIME-WINDOW FILTER WITH 30-MINUTE TRAVEL BUFFER:
 - Map intent.time_window to a target datetime window (use simple heuristics: "now" -> next 60 min, "today_morning" -> 8-12 today, etc.)
 - For each provider, find any available_slot that falls within the window, with 30-min buffer before and after.
 - Compute availability_status: available_now / next_slot_within_2h / next_slot_today / tomorrow_or_later
6. SCHEDULING INTELLIGENCE: if fewer than 3 candidates remain:
 - Take top 5 providers (ignoring time-window) by rating
 - Find their next available slot
 - Add to alternates with alternate_slot_reason = "Earliest available is {slot} instead of {requested}"
7. PRE-RANK SCORE: sort by (availability_weight + rating + (1 - risk_score) + (1 - current_workload))

availability_weight: available_now=1.0, within_2h=0.8, today=0.5, later=0.2
8. PHONE MASKING: phone_masked = "+92 XXX-XXX-" + phone[-4:]; full phone never in DiscoveryResult.
9. Take top 8.
10. Build ProviderCandidate objects with all fields populated from provider data + computed distance_km, availability_status, alternate_slot_reason (only on alternates).

NO-MATCH REASONS:

- If after service/gender/complexity filters zero remain: no_match_reason = "No available in matching your needs."
- If after time filter zero remain AND alternates also zero: no_match_reason = "Nobody is free in your time window; consider flexible timing."

```

async def run(intent: Intent, user_location: Optional[Tuple[float,float]] = None) -> DiscoveryResult:
log.info("agent_start", agent="discovery")
t0 = time.monotonic()
# ... apply algorithm above ...
result = DiscoveryResult(candidates=candidates, alternates=alternates, no_match_reason=reason)
log.info("agent_end", agent="discovery", duration_ms=int((time.monotonic()-t0)*1000),
n_candidates=len(candidates), n_alternates=len(alternates))
return result

```

WIRE INTO ORCHESTRATOR:

- After intent agent, if intent.confidence >= 0.7, call discovery_agent.run(intent, req.user_location)
- Add to response: discovery: DiscoveryResult
- If discovery.no_match_reason is set, set a flag in the response and skip downstream agents.

TESTS (app/tests/test_discovery_agent.py):

- a) Exact neighborhood match: intent for G-13 returns only G-13 + nearby providers
- b) Emergency intent prioritizes available_now providers
- c) Gender filter: gender_preference="female" returns only female providers
- d) Complexity filter: "complex" excludes years_experience < 5
- e) Alternate-slot path triggers when filtered set < 3
- f) Empty result returns no_match_reason populated
- g) Phone is masked (no_full_phone in candidates)

VERIFY:

1. Is app/data/providers.json -> 100 entries, valid JSON
2. poetry run pytest app/tests/test_discovery_agent.py -> all 7 tests pass
3. curl -X POST localhost:8080/orchestrate -d '{"text":"Plumber chahiye G-13 mein kal subah","user_id":"u_test"}' -> discovery.candidates is non-empty, every candidate has neighborhood=="G-13" or distance_km < 5
4. curl -X POST localhost:8080/orchestrate -d '{"text":"female bridal beautician F-7 mein this Friday","user_id":"u_test"}' -> all candidates have gender=="female" and "beautician" in service_categories

End-of-day verification

- Intent Agent eval shows ≥92% service_type and ≥85% job_complexity accuracy.
- Low-confidence intents (e.g. "mere ghar pe plumber bhejo") return confidence < 0.7 AND a populated clarification_question.
- providers.json has exactly 100 entries; covers all 14 neighborhoods.
- Discovery Agent returns ≤8 candidates ordered sensibly; alternates populated when filter is tight.
- Phone numbers are masked in all DiscoveryResult outputs.
- curl /orchestrate with a Pakistani Urdu sentence returns plausible intent + provider list.

- Antigraavity screenshots saved to docs/antigravity-trace/day2/: Plan Artifact for Intent Agent, browser recording of the eval script running.

■ *At 8pm, post in WhatsApp: 'Intent + Discovery are real. Pull and import app.agents.intent_agent and app.agents.discovery_agent.' Backend Workflows will use these in the /orchestrate handler.*

06 · DAY 4 · MAY 17

Day 3 · Ranking + Pricing agents

Two agents today — Ranking (six-factor scoring, Gemini 3 Pro) and the new-in-v2 Pricing Agent (transparent line-item quote, Gemini Flash). By noon, hand off to Backend Workflows for integration into the orchestrator.

Morning · the Ranking Agent (six-factor)

The Ranking Agent is the most visible LLM in the demo — its Urdu reasoning types out on screen. Build it with deterministic Python scoring + Gemini 3 Pro for natural-language reasoning. **DO NOT** have the LLM compute numbers; the LLM writes prose about numbers the Python already computed.

PROMPT · DAY 3 — RANKING AGENT (PASTE INTO ANTIGRAVITY)

ROLE: Build the Ranking Agent using Gemini 3 Pro. It scores candidates across 6 factors using urgency-weighted profiles, picks a winner, and produces reasoning in BOTH Urdu and English.

FILES TO EDIT:

- app/agents/ranking_agent.py (replace stub)
- app/prompts/ranking.md (paste from Product teammate; do not edit content)
- app/tests/test_ranking_agent.py (NEW)

IMPLEMENTATION (app/agents/ranking_agent.py):

```
import json, time
from pathlib import Path
from typing import List
from google.adk.agents import LlmAgent
from google.adk.models import Gemini
from app.config import settings
from app.models import Intent, ProviderCandidate, RankingResult, FactorScores

_PROMPT = (Path(__file__).parent.parent / "prompts" / "ranking.md").read_text(encoding="utf-8")

_agent = LlmAgent(
    name="ranking",
    model=Gemini(model_name="gemini-3-pro-latest", temperature=0.4, api_key=settings.GEMINI_API_KEY),
    system_instruction=_PROMPT,
    response_mime_type="application/json",
)

URGENCY_WEIGHTS = {
    "emergency": {"distance":0.30,"availability":0.30,"on_time":0.20,"rating":0.15,"specialization":0.03,"risk":0.02},
    "high": {"distance":0.20,"availability":0.25,"on_time":0.20,"rating":0.20,"specialization":0.10,"risk":0.05},
    "normal": {"distance":0.15,"availability":0.10,"on_time":0.20,"rating":0.25,"specialization":0.20,"risk":0.10},
    "flexible": {"distance":0.10,"availability":0.05,"on_time":0.15,"rating":0.30,"specialization":0.30,"risk":0.10},
}
```



```

def _compute_factor_scores(c: ProviderCandidate, intent: Intent) -> FactorScores:
    distance = max(0.0, 1.0 - (c.distance_km / 8.0))
    availability_map = {"available_now": 1.0, "next_slot_within_2h": 0.8, "next_slot_today": 0.5,
                        "tomorrow_or_later": 0.2}
    availability = availability_map.get(c.availability_status, 0.0)
    rating = c.rating / 5.0
    on_time = c.on_time_score
    if intent.specialization_hint and intent.specialization_hint in c.specializations:
        specialization = 1.0
    elif intent.specialization_hint:
        specialization = 0.3
    else:
        specialization = 0.7
    risk = 1.0 - c.risk_score
    weights = URGENCY_WEIGHTS[intent.urgency]
    total = (distance*weights["distance"] + availability*weights["availability"]
            + on_time*weights["on_time"] + rating*weights["rating"]
            + specialization*weights["specialization"] + risk*weights["risk"])
    return FactorScores(distance=round(distance,2), availability=round(availability,2),
                        rating=round(rating,2), on_time=round(on_time,2),
                        specialization=round(specialization,2), risk=round(risk,2),
                        total=round(total,3))

async def run(intent: Intent, candidates: List[ProviderCandidate]) -> RankingResult:
    log.info("agent_start", agent="ranking", n_candidates=len(candidates))
    t0 = time.monotonic()

    if not candidates:
        return RankingResult(recommended_id=None, error="no_candidates")

    # Compute deterministic factor scores in Python
    factor_scores = {c.id: _compute_factor_scores(c, intent) for c in candidates}

    # Build LLM payload — pass the candidates + factor_scores; the LLM writes the reasoning.
    payload = {
        "intent": intent.model_dump(),
        "candidates": [c.model_dump() for c in candidates],
        "factor_scores_precomputed": {cid: fs.model_dump() for cid, fs in factor_scores.items()},
    }

```

```

try:
    raw = await _agent.invoke(json.dumps(payload, ensure_ascii=False))
    data = json.loads(raw)
except Exception as e:
    log.error("ranking_llm_failure", error=str(e))
    # Fallback: pick top by total score, generic reasoning
    sorted_cands = sorted(candidates, key=lambda c: factor_scores[c.id].total, reverse=True)
    top = sorted_cands[0]
    return RankingResult(
        recommended_id=top.id,
        top_three_ids=[c.id for c in sorted_cands[:3]],
        factor_scores=factor_scores,
        reasoning_english=f"{top.name} ranks highest with {top.rating} rating, {top.distance_km}km away.",
        reasoning_urdu=f"{top.name} sab se behtar hai — {top.rating} rating, sirf {top.distance_km} km door.",
        tradeoffs="",
        confidence="medium",
    )

# Validate, but trust the LLM's recommended_id and reasoning; use OUR computed factor_scores
result = RankingResult(
    recommended_id=data.get("recommended_id"),
    top_three_ids=data.get("top_three_ids", []),
    factor_scores=factor_scores, # always our compute, never trust LLM here
    reasoning_english=data.get("reasoning_english", ""),
    reasoning_urdu=data.get("reasoning_urdu", ""),
    tradeoffs=data.get("tradeoffs", ""),
    confidence=data.get("confidence", "medium"),
)

# Validate: recommended_id must be in candidates
if result.recommended_id not in {c.id for c in candidates}:
    sorted_cands = sorted(candidates, key=lambda c: factor_scores[c.id].total, reverse=True)
    result.recommended_id = sorted_cands[0].id
# Validate: reasoning must contain at least 3 digits
if sum(ch.isdigit() for ch in result.reasoning_english) < 3:
    log.warn("ranking_reasoning_lacks_specificity", english=result.reasoning_english[:80])

log.info("agent_end", agent="ranking", duration_ms=int((time.monotonic()-t0)*1000),
    recommended_id=result.recommended_id)
return result

WIRE INTO ORCHESTRATOR after Discovery:
- ranking = await ranking_agent.run(intent, discovery.candidates)

TESTS (app/tests/test_ranking_agent.py):
- a) Emergency intent uses emergency weights (test by mocking candidates and checking factor_score totals)
- b) Reasoning_english contains at least 3 digits
- c) Reasoning_urdu is non-empty and is detectably Urdu (contains at least one of: hai, behtar, sirf, door, mein, ke, ki, ka)
- d) top_three_ids has exactly 3 entries when ≥3 candidates exist
- e) factor_scores has an entry for every candidate
- f) If candidates is empty, returns recommended_id=None and error=="no_candidates"

```

VERIFY:

1. poetry run pytest app/tests/test_ranking_agent.py -> 6 pass
2. curl -X POST localhost:8080/orchestrate -d '{"text": "Mujhe G-13 mein kal subah AC theek karwana hai", "user_id": "u_test"}' -> ranking.recommended_id non-null, factor_scores populated for all candidates, reasoning_urdu sounds Pakistani (sample manually)

Afternoon · the Pricing Agent (new in v2)

The Pricing Agent sits between Ranking and Booking. The math is deterministic Python; the LLM only writes the natural-language explanation. The output is a PriceQuote with line_items the mobile UI displays before the user can confirm.

PROMPT · DAY 3 — PRICING AGENT (PASTE INTO ANTIGRAVITY)

ROLE: Build the new Pricing Agent (added in v2 per the official Challenge 2 spec). It produces a transparent line-item quote BEFORE the user confirms the booking.

FILES TO EDIT/CREATE:

- app/agents/pricing_agent.py (replace stub)
- app/prompts/pricing.md (paste from Product team; pricing math here, only natural-language goes through LLM)
- app/tests/test_pricing_agent.py (NEW)

PRICING MATH (deterministic Python — runs BEFORE the LLM is called):

```
from typing import List
import uuid
from datetime import datetime, timedelta
from app.models import Intent, ProviderCandidate, PriceLineItem, PriceQuote

COMPLEXITY_HOURS = {"basic": 1.0, "intermediate": 1.5, "complex": 2.5}
COMPLEXITY_MULTIPLIER = {"basic": 1.0, "intermediate": 1.25, "complex": 1.6}
URGENCY_SURCHARGE = {"emergency": 500, "high": 200, "normal": 0, "flexible": -100}

def _compute_line_items(intent: Intent, provider: ProviderCandidate,
market_demand: float, is_first_booking: bool) -> tuple[List[PriceLineItem], int]:
    items = []
    # 1. Base visit fee
    items.append(PriceLineItem(
        label_english="Visit fee",
        label_urdu="Visit fee",
        amount_pkr=provider.base_visit_fee_pkr,
        kind="base"))
```

```
# 2. Distance fee (free first 3km, then 50 PKR/km)
extra_km = max(0, provider.distance_km - 3.0)
distance_fee = int(extra_km * 50)
if distance_fee > 0:
    items.append(PricingItem(
        label_english=f"Distance ({extra_km:.1f} km)",
        label_urdu=f"Distance ({extra_km:.1f} km)",
        amount_pkr=distance_fee,
        kind="base"))

# 3. Estimated service fee
hours = COMPLEXITY_HOURS[intent.job_complexity]
service_fee = int(provider.rate_per_hour_pkr * hours)
items.append(PricingItem(
    label_english=f"Service time (~{hours}h)",
    label_urdu=f"Service time (~{hours} ghante)",
    amount_pkr=service_fee,
    kind="base"))

# Subtotal so far
subtotal_pre = sum(i.amount_pkr for i in items)

# 4. Complexity multiplier (only if not basic)
multiplier = COMPLEXITY_MULTIPLIER[intent.job_complexity]
if multiplier > 1.0:
    adjustment = int(subtotal_pre * (multiplier - 1.0))
    items.append(PricingItem(
        label_english=f"Complexity adjustment ({intent.job_complexity})",
        label_urdu=f"Complexity ({intent.job_complexity})",
        amount_pkr=adjustment,
        kind="multiplier"))

# 5. Urgency surcharge (only if non-zero)
surcharge = URGENCY_SURCHARGE[intent.urgency]
if surcharge != 0:
    items.append(PricingItem(
        label_english=f"Urgency ({intent.urgency})",
        label_urdu=f"Urgency ({intent.urgency})",
        amount_pkr=surcharge,
        kind="addon"))
```

```

# 6. Surge factor
if market_demand > 0.7:
    surge_pct = 0.20
elif market_demand > 0.5:
    surge_pct = 0.10
else:
    surge_pct = 0.0
if surge_pct > 0:
    running = sum(i.amount_pkr for i in items)
    surge_amount = int(running * surge_pct)
    items.append(PriceLineItem(
        label_english=f"Demand surge (+{int(surge_pct*100)}%)",
        label_urdu=f"Demand surge (+{int(surge_pct*100)}%)",
        amount_pkr=surge_amount,
        kind="surge"))

# 7. Loyalty discount (welcome on first booking)
if is_first_booking:
    running = sum(i.amount_pkr for i in items)
    discount = max(-200, min(-50, -int(running * 0.05)))
    items.append(PriceLineItem(
        label_english="Welcome discount",
        label_urdu="Welcome discount",
        amount_pkr=discount,
        kind="discount"))

total = sum(i.amount_pkr for i in items)
total_rounded = round(total / 10) * 10 # round to nearest 10 PKR

return items, total_rounded

async def run(intent: Intent, provider: ProviderCandidate,
    market_demand: float = 0.65, is_first_booking: bool = True) -> PriceQuote:
    log.info("agent_start", agent="pricing")
    t0 = time.monotonic()

    line_items, total = _compute_line_items(intent, provider, market_demand, is_first_booking)
    range_low = int(total * 0.9)
    range_high = int(total * 1.15)
    if range_high - range_low < 250:
        range_high = range_low + 250

    # Call LLM only for the natural-language explanation (English + Urdu + fairness_note)
    payload = {
        "intent": intent.model_dump(),
        "provider": provider.model_dump(),
        "market_demand": market_demand,
        "is_first_booking": is_first_booking,
        "line_items": [i.model_dump() for i in line_items],
        "estimated_total_pkr": total,
        "estimated_range_pkr": [range_low, range_high],
    }

```

```

try:
raw = await _llm_agent.invoke(json.dumps(payload, ensure_ascii=False))
data = json.loads(raw)
explanation_en = data.get("explanation_english", "")
explanation_ur = data.get("explanation_urdu", "")
fairness = data.get("fairness_note", "")
except Exception as e:
log.error("pricing_llm_failure", error=str(e))
explanation_en = f"Your estimate is PKR {range_low:,.} - {range_high:,.} based on the line items below."
explanation_ur = f"Aapka estimate PKR {range_low:,.} - {range_high:,.} hai, neeche di gayi tafseelat ke mutaabiq."
fairness = f"Provider receives PKR {int(total * 0.85):,.} after platform fee."

quote = PriceQuote(
quote_id=f"QT-{datetime.now().strftime('%Y%m%d')}-{uuid.uuid4().hex[:6].upper()}",
line_items=line_items,
subtotal_pkr=sum(i.amount_pkr for i in line_items),
estimated_total_pkr=total,
estimated_range_pkr=(range_low, range_high),
explanation_english=explanation_en,
explanation_urdu=explanation_ur,
fairness_note=fairness,
expires_at=(datetime.now() + timedelta(minutes=15)).isoformat() + "+05:00",
)

log.info("agent_end", agent="pricing", duration_ms=int((time.monotonic()-t0)*1000),
estimated_total_pkr=total)
return quote

```

WIRE INTO ORCHESTRATOR:

- After Ranking, look up the recommended candidate, then:

```
pricing = await pricing_agent.run(intent, recommended_provider, market_demand=0.65,
is_first_booking=True)
```
- Add to response: pricing: PriceQuote

TESTS:

- a) Complex job costs more than basic for same provider
- b) Emergency adds 500 surcharge line item
- c) Surge applied when market_demand > 0.5
- d) First booking has welcome_discount line item with negative amount
- e) Every line_item has both English and Urdu labels
- f) estimated_range_pkr is always at least 250 PKR wide

VERIFY:

1. poetry run pytest app/tests/test_pricing_agent.py -> 6 pass
2. curl /orchestrate with an AC repair intent -> pricing has multiple line_items, estimated_range_pkr makes sense, explanation_urdu sounds natural

End-of-day verification

- Ranking Agent factor_scores total to a sensible weighted sum.

- Ranking Agent reasoning contains at least 3 specific numbers.
- Ranking Agent reasoning_urdu is natural Pakistani Urdu (not translated English).
- Pricing Agent produces 4-7 line_items in the correct order.
- Pricing Agent range is always ≥ 250 PKR wide.
- Pricing Agent surge mentioned only when market_demand > 0.5.
- Both agents wired as STUBS in the orchestrator: orchestrate() calls them but returns placeholder booking/followup.
- Three Antigravity Plan Artifacts saved for today's agents.

At noon, post in WhatsApp: 'Ranking + Pricing are importable. Pull now. Booking now needs the PriceQuote from pricing_agent.run().' Backend Workflows needs this to build the Booking Agent properly.

07 · DAY 5 · MAY 18

Day 4 · eval loop + schema freeze

Run the eval harness against all 50 examples. Iterate the Intent Agent prompt until 92/85 targets are met. Freeze schemas for Workflows.

Morning · eval harness + full 50-example run

Today you harden the eval harness and run it against the full 50-example dataset the Product/Urdu teammate finished on Day 2. The target: 92%+ service_type, 85%+ job_complexity, 75%+ confidence calibration. You iterate the prompt if it misses; you DO NOT touch the dataset.

PROMPT · DAY 4 — INTENT AGENT EVAL + ITERATION

Run the eval harness at backend/scripts/eval_intent.py against the full 50-example dataset at backend/data/intent_examples.jsonl. Report accuracy; iterate the prompt until targets are met.

STEP 1 — Verify the eval script works:

```
cd backend
```

```
poetry run python scripts/eval_intent.py --dataset data/intent_examples.jsonl --report eval_report.md
```

Open eval_report.md. Check it contains:

- service_type_accuracy: float
- job_complexity_accuracy: float
- confidence_calibration: float
- failure_cases: list of {input, expected, actual}

STEP 2 — Check current accuracy against targets:

service_type accuracy target : >= 92%

job_complexity accuracy target : >= 85%

confidence calibration target : >= 75%

STEP 3 — If service_type accuracy < 92%:

Read the top 5 failure_cases.

For each, diagnose the failure in one sentence (e.g., "Model returns 'plumber' for 'geyser wala' — the word 'geyser' should map to geyser_technician").

Apply ONE targeted edit to backend/agents/prompts/intent_agent_system.md per round.

Re-run eval after each edit. Stop when target is met OR you have done 5 rounds.

If you cannot hit 92% in 5 rounds, report the plateau and the residual failure pattern.

STEP 4 — If job_complexity accuracy < 85%:

Same process: diagnose top failures, apply one edit per round.

STEP 5 — Save the final eval_report.md. It will be embedded in the README (Section 8).

Commit:

```
git add eval_report.md backend/agents/prompts/intent_agent_system.md
```

```
git commit -m "eval: Intent Agent reaches % service_type, % complexity"
```

```
git push
```

After the eval session, POST the final numbers in team WhatsApp so Product/Urdu can update their README draft.

Afternoon · Pydantic schema audit

The Workflows teammate (Member 4) is building against your Pydantic schemas today. A schema change tonight breaks their Booking and orchestrator code. Freeze the schemas with a version comment and run the full test suite.

PROMPT · DAY 4 — SCHEMA FREEZE + TEST SUITE

Audit and freeze the shared Pydantic schemas in `backend/app/schemas/`. The Workflows teammate is importing these; any change after today requires a WhatsApp heads-up first.

AUDIT CHECKLIST:

1. Open each schema file: `intent.py`, `provider_candidate.py`, `price_quote.py`, `discovery_result.py`, `ranking_result.py`.
 2. For each schema, confirm:
 - a) All fields used by the system prompts in `backend/agents/prompts/` are present.
 - b) Optional fields have sensible defaults (not bare `None` without `Optional[]`).
 - c) Literal type constraints are in place for enum-like fields (e.g., `urgency: Literal["emergency", "high", "normal", "flexible"]`).
 - d) The schema can round-trip through `json.dumps/json.loads` without loss.
 3. Add a version comment at the top of each schema file:
`# Schema version: 1.0 — FROZEN as of Day 4. Coordinate with Backend Workflows before changing.`
 4. Run the full test suite:
`cd backend`
`poetry run pytest tests/ -v`
 All tests must pass. Fix any that fail before committing.
 5. Run a type-check pass:
`poetry run mypy app/ --ignore-missing-imports`
 Fix any type errors (not just warnings).
 6. Commit:
`git commit -am "schemas: freeze v1.0 — all tests passing, mypy clean"`
`git push`
- POST IN WHATSAPP: "Backend Agents: schemas frozen at v1.0. Intent eval: service_type %, complexity %. Workflows teammate can safely import."

End-of-day verification

- Eval harness runs without errors against all 50 examples.
- Final service_type accuracy $\geq 92\%$.
- Final job_complexity accuracy $\geq 85\%$.
- `eval_report.md` committed to repo.
- All five schema files have the version freeze comment.
- Full test suite passes: `poetry run pytest tests/ -v`.

- mypy shows 0 type errors.
- Schema freeze posted in team WhatsApp.

08 · DAY 6 · MAY 19

Day 5 · polish, tests, Docker

No new agents. All tests pass. Docker image builds. Infra can deploy.

Full day · polish, test coverage, Docker verify

No new agents today. This session runs tests, verifies Docker, and hands off cleanly to the Infra teammate.

PROMPT · DAY 5 — FINAL POLISH AND HANDOFF

ROLE: Final polish day. No new agents. Goal: every code path is robust, all tests pass, the production Docker image is tagged v1.0.0 and pushed to Artifact Registry, and the Infra teammate has everything they need to ship.

TASKS (in order, no LLM needed except for content review):

1. Run the full test suite. Every test must pass.

```
poetry run pytest -v
```

If any fail, fix them. Do not skip.

2. Run ruff and fix all linting issues.

```
poetry run ruff check . --fix
```

```
poetry run ruff format .
```

3. Run the eval one more time.

```
poetry run python scripts/eval_intent.py
```

Confirm: service_type accuracy >= 92%, job_complexity accuracy >= 85%.

4. End-to-end smoke test against deployed Cloud Run (Infra owns the URL).

```
curl https://health -> 200
```

Run through 5 different real Urdu sentences from the dataset, verify every response is sensible.

5. Update version. In app/main.py change version to "1.0.0". In Dockerfile add LABEL version="1.0.0".

6. Build and tag the production image.

```
docker build -t asia-south1-docker.pkg.dev/bulao-hackathon/bulao/backend:v1.0.0 -t
```

```
asia-south1-docker.pkg.dev/bulao-hackathon/bulao/backend:latest .
```

```
docker push asia-south1-docker.pkg.dev/bulao-hackathon/bulao/backend:v1.0.0
```

```
docker push asia-south1-docker.pkg.dev/bulao-hackathon/bulao/backend:latest
```

7. Update README.md (final version).

Sections (each 1-2 paragraphs):

- Overview (what Bulao is)
- Architecture (link to /docs/diagrams/architecture.png)
- Six agents (one paragraph each)
- Setup & run locally
- Deploy to Cloud Run
- Tests & eval
- Environment variables
- Data files & schemas
- Known limitations
- License (MIT)

8. Generate the final canned demo response.

Use a real /orchestrate call: 'Mujhe G-13 mein kal subah AC theek karwana hai'

Capture the response, save to app/data/demo_canned_response.json.

Verify by running DEMO_MODE=true ... and confirming the response matches.

9. Final handoff package for Infra (push these to the repo):

- The production image tag (v1.0.0)
- The Firestore export JSON (gcloud firestore export -> share GCS URI)
- A short HANDOFF.md listing: image URI, deployed Cloud Run URL, env vars required, who owns what after submission

10. Final Antigravity screenshot capture (for the Trace deliverable):

- One Manager View screenshot showing the full repo
- One Plan Artifact screenshot for a representative agent
- The test suite output screenshot showing all green

Save to docs/antigravity-trace/day5/.

VERIFY:

1. poetry run pytest -v -> all green
2. ruff check . -> no issues
3. docker images | grep bulao-hackathon/bulao/backend:v1.0.0 -> exists locally AND in Artifact Registry
4. curl https://health on the v1.0.0 image -> {"status":"ok","version":"1.0.0"}
5. README is complete; HANDOFF.md is committed; Infra teammate has read it.

OUTPUT: Antigravity reports back with the test summary, the image digest, and a one-line confirmation that the handoff is complete.

End-of-day verification

- poetry run pytest tests/ -v all green.
- All 4 agents (Intent, Discovery, Ranking, Pricing) return valid Pydantic models.
- Discovery Agent handles the alternate-slot path for < 3 candidates.
- Intent Agent handles the clarification path when confidence < 0.7.
- Docker image builds: docker build -t bulao-backend .
- README Section 8 data (eval accuracy numbers) confirmed accurate.

■ Final commit tagged: `git tag agents-v1.0.0`

APPENDIX A

The /orchestrate response contract.

This is the exact JSON shape your /orchestrate endpoint must return by Day 1 evening (as stubs) and Day 3 noon (with real Ranking + Pricing data). Mobile parses this shape; any field name change after Day 1 costs Mobile half a day.

```
// The API the Mobile teammate is wiring to.
// All endpoints accept and return JSON. Errors come as {error: str, code: int} with HTTP 4xx/5xx.

GET /health
-> 200 {"status":"ok", "version":"<semver>", "commit":"<git-sha>"}

POST /orchestrate
Body: {
  text: str,           // user input – Urdu / Roman Urdu / English / mixed
  user_id: str,         // stable per-user identifier from the mobile app
  user_location: [lat: float, lng: float] // optional
}
-> 200 {
  intent: Intent,
  discovery: DiscoveryResult,
  ranking: RankingResult,
  pricing: PriceQuote,
  booking_preview: {    // not yet committed; user must call /book to commit
    provider_id: str,
    scheduled_time: str,
    expected_total_pkr: int
  },
  followup_planned: {
    reminder_at: str,    // ISO datetime
    checkin_at: str
  },
  warnings: []          // list of degraded-mode warnings, e.g. "intent_fallback_keyword"
}
-> 200 with needs_clarification=true if intent.confidence < 0.7:
  { intent, needs_clarification: true, clarification_question: str }
```

```
-> 200 with no_match_reason if discovery returns empty:
  { intent, discovery: {candidates:[], alternates:[], no_match_reason: str},
    user_message_urdu: str, user_message_english: str }
```

POST /book

```
Body: {
  quote_id: str,
  user_id: str,
  user_name: str | null,
  intent: Intent,
  provider_id: str,
  accepted_quote: PriceQuote
}
```

```
-> 200 Booking
```

```
-> 409 {"error":"quote_expired"} | {"error":"slot_taken"}
```

POST /lifecycle

```
Body: {booking_id: str, new_status: "en_route"|"arrived"|"in_progress"|"completed"}
```

```
-> 200 Booking (updated)
```

POST /dispute

```
Body: {
  booking_id: str,
  user_complaint: str,
  service_checklist: [{item: str, status: "yes"|"no"}],
  rating: int    // 1-5
}
```

```
-> 200 FollowupResult (mode="dispute") with classification, resolution, refund_pkr, urdu/english
messages
```

APPENDIX B

Canonical agent code pattern.

Every LLM-backed agent follows the same structure: load the .md prompt once at module import, call ADK, parse JSON, validate with Pydantic, retry once on failure, fall back to a safe default. The Discovery Agent omits the LLM call but follows the same function signature.

```
# backend/app/agents/intent_agent.py – canonical structure
# All four agents follow this pattern.

import json
from pathlib import Path
from typing import Optional

from google.adk.agents import LlmAgent
from google.adk.models.lite_llm import LiteLlm
from pydantic import ValidationError

from app.schemas.intent import Intent
from app.config import settings

_PROMPT_PATH = Path(__file__).parent.parent / "agents" / "prompts" / "intent_agent_system.md"
_PROMPT_TEXT: Optional[str] = None

def _load_prompt() -> str:
    global _PROMPT_TEXT
    if _PROMPT_TEXT is None:
        _PROMPT_TEXT = _PROMPT_PATH.read_text()
    return _PROMPT_TEXT

async def run(user_text: str) -> Intent:
    """Extract structured intent from natural language text."""
    model = LiteLlm(model="gemini/gemini-flash-latest", temperature=0.1)
    agent = LlmAgent(
        name="intent_agent",
```



```

        model=model,
        instruction=_load_prompt(),
    )

    result = await agent.run(user_text)
    raw_text = result.text.strip()

    # Strip markdown fences if present
    if raw_text.startswith("```"):
        raw_text = raw_text.split("```")[1]
        if raw_text.startswith("json"):
            raw_text = raw_text[4:].strip()

    try:
        data = json.loads(raw_text)
        return Intent.model_validate(data)
    except (json.JSONDecodeError, ValidationError) as e:
        # Retry once with explicit JSON reminder
        retry_result = await agent.run(
            f"{user_text}\n\nReturn ONLY valid JSON. Previous error: {e}"
        )
    try:
        data = json.loads(retry_result.text.strip())
        return Intent.model_validate(data)
    except Exception:
        # Fallback intent with confidence=0.3 asking for clarification
        return Intent(
            service_type=_keyword_fallback(user_text),
            confidence=0.3,
            clarification_question="Aap ne kya kaam karwana hai?",
        )

def _keyword_fallback(text: str) -> str:
    """Very naive keyword-match for the absolute worst-case fallback."""
    text_lower = text.lower()
    if any(w in text_lower for w in ["ac", "cooling", "heat", "heater"]):
        return "ac_technician"
    if any(w in text_lower for w in ["water", "pipe", "tap", "geyser", "paani"]):
        return "plumber"
    if any(w in text_lower for w in ["light", "bijli", "electric", "wiring"]):
        return "electrician"
    return "appliance_repair" # safest fallback

# backend/app/agents/discovery_agent.py – pure Python, no LLM
import math
from typing import Optional

from app.schemas.intent import Intent
from app.schemas.discovery_result import DiscoveryResult, ProviderCandidate
from app.data.provider_store import load_providers

def run(intent: Intent, user_lat: Optional[float] = None,
        user_lng: Optional[float] = None) -> DiscoveryResult:
    """Filter, schedule-check, and pre-rank providers for the given intent."""
    providers = load_providers()

    # 1. Service filter

```

```

candidates = [p for p in providers if intent.service_type in p.service_categories]

# 2. Specialization preference (boost, not filter)
# handled by pre-rank score below

# 3. Location filter (5km radius)
neighborhood_center = _get_neighborhood_center(intent.location)
if neighborhood_center and not (user_lat and user_lng):
    user_lat, user_lng = neighborhood_center
if user_lat and user_lng:
    candidates = [p for p in candidates
                   if _haversine(user_lat, user_lng, p.lat, p.lng) <= 5.0]

# 4. Gender filter
if intent.gender_preference in ("male", "female"):
    candidates = [p for p in candidates if p.gender == intent.gender_preference]

# 5. Complexity filter
if intent.job_complexity == "complex":
    candidates = [p for p in candidates
                   if p.years_experience >= 5 and _has_specialization(p, intent)]
elif intent.job_complexity == "intermediate":
    candidates = [p for p in candidates if p.years_experience >= 2]

# 6. Time-window + travel buffer
available = [p for p in candidates if _time_matches(p, intent)]

# 7. Alternate slot path if < 3 available
alternates = []
if len(available) < 3:

    alternates = _find_alternates(candidates, intent, exclude=available)

# 8. Pre-rank and cap at 8
available.sort(key=lambda p: _pre_rank(p, intent), reverse=True)
available = available[:8]

return DiscoveryResult(
    candidates=available,
    alternates=alternates[:3],
    no_match_reason=_no_match_reason(intent) if not available else None,
)
```

APPENDIX C

Generate the 100-provider mock dataset.

Paste this prompt into Antigravity on Day 1 afternoon to generate backend/data/providers.json. The dataset must match the v2 schema exactly — Ranking and Pricing agents read these fields directly.

Generate a complete backend/data/providers.json file with exactly 100 Pakistani service providers.

SCHEMA (each provider):

```
{
  "id": "prov_NNN",
  "name": "<authentic Pakistani full name>",
  "service_categories": ["<one or more of 10 service types>"],
  "specializations": ["<relevant skills, 0-4>"],
  "lat": <float>, "lng": <float>,
  "neighborhood": "<one of 14 neighborhoods below>",
  "city": "<Islamabad|Rawalpindi|Karachi|Lahore>",
  "rating": <float 3.5-5.0>,
  "completed_jobs": <int 20-300>,
  "completed_jobs_by_neighborhood": {"<nbhd>": <int>, ...},
  "on_time_score": <float 0.75-1.0>,
  "cancellation_rate": <float 0.0-0.15>,
  "review_recency_days": <int 1-90>,
  "risk_score": <float 0.02-0.30>,
  "current_workload": <float 0.0-1.0>,
  "available_slots": ["<ISO8601 datetime +05:00>", ...],
  "phone": "+923XX-XXXXXXX",
  "gender": "male|female",
  "years_experience": <int 1-25>,
  "languages": ["urdu", "english"|<punjabi>|...],
  "base_visit_fee_pkr": <int 300-800>,
  "rate_per_hour_pkr": <int 800-2500>,
  "price_range_pkr": [<int>, <int>],
  "verified": true|false
}
```

NEIGHBORHOODS TO COVER (at least 5 providers each):

G-9, G-10, G-11, G-13, F-6, F-7, F-8, F-10, F-11,
Bahria Town Phase 4, DHA Islamabad Phase 2,
Saddar Rawalpindi, DHA Karachi Phase 6, DHA Lahore Phase 5

SERVICE TYPE DISTRIBUTION across 100 providers:

plumber: 15, electrician: 18, ac_technician: 20,
geyser_technician: 10, carpenter: 8, painter: 7,
beautician: 10, tutor: 6, appliance_repair: 4, gas_leak_specialist: 2

OTHER DISTRIBUTION:

Female providers: at least 15 (concentrate in beautician + tutor)

Verified: at least 80

Available slots: each provider has 3-6 slots in the next 7 days

Use realistic Pakistani names (male and female).

Phone numbers: +923XX-XXXXXXX format with valid PK prefixes (310, 311, 315, 321, 333, 336, 345).

Lat/lng must correspond to real neighborhood locations in their respective cities.

Save as backend/data/providers.json.

Validate with: cd backend && python -c "import json; data=json.load(open('data/providers.json'));
print(len(data), 'providers')"

Expected output: 100 providers